

# Visual Exploratory Data Analysis (VisEDA): PART B

## Video Exploratory Data Analysis

A Guide to Temporal & Visual Video Analysis

Isaac Osei Agyemang

Version 1.0.0

2026

# Preface

---

VisEDA (Visual Exploratory Data Analysis) was built to bring the rigour of tabular EDA tools to visual datasets. The library's first module — ImageEDA — established a comprehensive framework for understanding image datasets. VideoEDA extends this framework into the temporal dimension, treating videos not merely as sequences of images but as rich, dynamic signals with properties that cannot be captured by analysing individual frames in isolation.

Video datasets present unique challenges. A dataset of 10,000 videos may contain 5 million frames — far too many to inspect manually. Quality issues manifest differently: a sharp first frame may be followed by a blurry middle section caused by camera motion. Scene changes create abrupt discontinuities. Lighting flicker introduces temporal instability invisible in per-frame statistics. VideoEDA is designed to surface all of these issues automatically.

This documentation covers VideoEDA version 1.0.0 in complete detail: every parameter, every metric, every visualisation, and every practical usage pattern. The source code, tests, and CLI examples are all reflected here.

Isaac Osei Agyemang

May 2026

# Table of Contents

---

|                                                 |           |
|-------------------------------------------------|-----------|
| <b>Preface .....</b>                            | <b>2</b>  |
| <b>Table of Contents .....</b>                  | <b>3</b>  |
| <b>Chapter 1 — Introduction .....</b>           | <b>6</b>  |
| 1.1 What is VideoEDA?.....                      | 6         |
| 1.2 Why Video EDA Matters .....                 | 6         |
| 1.3 Design Philosophy .....                     | 6         |
| 1.4 Installation .....                          | 6         |
| 1.5 Supported Video Formats .....               | 7         |
| <b>Chapter 2 — Quick Start .....</b>            | <b>8</b>  |
| 2.1 Single Video .....                          | 8         |
| 2.2 Video Dataset from Directory .....          | 8         |
| 2.3 NumPy Array Videos.....                     | 9         |
| 2.4 CLI Usage .....                             | 9         |
| <b>Chapter 3 — The VideoEDA Class .....</b>     | <b>10</b> |
| 3.1 Constructor Parameters .....                | 10        |
| 3.2 load() — File Loading.....                  | 10        |
| 3.3 load_arrays() — Array Loading.....          | 11        |
| 3.4 summary() — Statistics Dictionary.....      | 11        |
| 3.5 get_record() and get_frames() .....         | 12        |
| 3.6 temporal_profile().....                     | 12        |
| 3.7 pairwise_video_distances() .....            | 12        |
| 3.8 report() .....                              | 13        |
| <b>Chapter 4 — The VideoRecord Object .....</b> | <b>14</b> |
| 4.1 Identity Fields .....                       | 14        |
| 4.2 Spatial Fields.....                         | 14        |
| 4.3 Temporal Fields.....                        | 14        |
| 4.4 Quality Fields.....                         | 14        |
| 4.5 Motion Fields.....                          | 15        |
| 4.6 Colour Fields.....                          | 15        |
| <b>Chapter 5 — What Is Analysed .....</b>       | <b>16</b> |
| 5.1 Frame Sampling Strategy .....               | 16        |
| 5.2 Spatial Properties .....                    | 16        |
| 5.3 Brightness .....                            | 16        |
| 5.4 Contrast.....                               | 17        |
| 5.5 Sharpness and Blur Detection.....           | 17        |
| 5.6 Noise and Compression.....                  | 17        |

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| 5.7 Frame Difference and Motion Intensity.....                   | 17        |
| 5.8 Motion Blur Detection.....                                   | 18        |
| 5.9 Scene Change Detection .....                                 | 18        |
| 5.10 Temporal Consistency.....                                   | 18        |
| 5.11 Colour Analysis .....                                       | 19        |
| <b>Chapter 6 — Visualisations.....</b>                           | <b>20</b> |
| 6.1 plot_dataset() — Dataset Dashboard.....                      | 20        |
| Row 0: Dataset Overview and Label Distribution .....             | 21        |
| Row 1: Spatial and Temporal Inventory .....                      | 21        |
| Row 2: Quality Metrics .....                                     | 21        |
| Row 3: Motion and Temporal Consistency.....                      | 21        |
| Row 4: Relationships and Colour .....                            | 22        |
| Row 5: Previews, Diversity, Resolution, Quality Flags .....      | 22        |
| 6.2 plot() — Single Video Dashboard.....                         | 22        |
| 6.3 plot_motion_profile() — Motion/Sharpness Profile .....       | 23        |
| 6.4 plot_pairwise_video_distances() — Diversity Heatmap.....     | 24        |
| 6.5 plot_videos_grid() — Video Thumbnail Grid .....              | 25        |
| 6.6 plot_temporal_statistics() — Temporal Deep Dive.....         | 26        |
| 6.7 plot_quality_summary() — Quality Summary .....               | 26        |
| 6.8 plot_frame_samples() — Frame Sample Grid.....                | 26        |
| <b>Chapter 7 — Interpreting Results and Common Findings.....</b> | <b>27</b> |
| 7.1 Red Flags to Watch For.....                                  | 27        |
| 7.2 Common Dataset Patterns .....                                | 27        |
| <b>Chapter 8 — Advanced Usage.....</b>                           | <b>28</b> |
| 8.1 Working with Large Datasets .....                            | 28        |
| 8.2 Accessing VideoRecord Objects .....                          | 28        |
| 8.3 Integrating with ML Pipelines.....                           | 29        |
| 8.4 Comparing Two Datasets .....                                 | 29        |
| <b>Chapter 9 — API Reference.....</b>                            | <b>31</b> |
| VideoEDA.__init__() .....                                        | 31        |
| VideoEDA.load().....                                             | 31        |
| VideoEDA.load_arrays() .....                                     | 31        |
| VideoEDA.summary().....                                          | 31        |
| VideoEDA.get_record(index).....                                  | 32        |
| VideoEDA.get_frames(index).....                                  | 32        |
| VideoEDA.temporal_profile(index).....                            | 32        |
| VideoEDA.pairwise_video_distances(max_videos) .....              | 32        |

|                                                               |           |
|---------------------------------------------------------------|-----------|
| VideoEDA.plot_dataset() .....                                 | 32        |
| VideoEDA.plot().....                                          | 32        |
| VideoEDA.plot_motion_profile() .....                          | 33        |
| VideoEDA.plot_pairwise_video_distances() .....                | 33        |
| VideoEDA.plot_videos_grid() .....                             | 33        |
| VideoEDA.plot_temporal_statistics() .....                     | 33        |
| VideoEDA.plot_quality_summary().....                          | 33        |
| VideoEDA.plot_frame_samples() .....                           | 33        |
| VideoEDA.report().....                                        | 34        |
| <b>Chapter 10 — HTML Report, CLI, and Test Workflow .....</b> | <b>35</b> |
| 10.1 HTML Report Architecture .....                           | 35        |
| 10.2 HTML Report Sections.....                                | 35        |
| 10.3 What the HTML Report Does Not Include .....              | 35        |
| 10.4 videda video CLI Reference.....                          | 36        |
| 10.5 CLI Examples.....                                        | 36        |
| 10.6 CLI vs Python API.....                                   | 36        |
| 10.7 testvideo.py Workflow .....                              | 37        |
| testvideo.py arguments .....                                  | 37        |
| 10.8 Test-Script Outputs and Report Behaviour.....            | 38        |
| 10.9 Troubleshooting .....                                    | 38        |
| <b>Chapter 11 — Roadmap .....</b>                             | <b>39</b> |
| Version 1.0.0 — Initial Release .....                         | 39        |
| Roadmap.....                                                  | 39        |
| <b>Appendix A — VideoRecord Fields .....</b>                  | <b>40</b> |
| <b>Appendix B — Glossary.....</b>                             | <b>41</b> |
| <b>Appendix C — Acknowledgements .....</b>                    | <b>42</b> |

# Chapter 1 — Introduction

---

## 1.1 What is VideoEDA?

VideoEDA is the video analysis module of VisEDA. It performs comprehensive exploratory data analysis on a single video file, a directory of videos, or a collection of in-memory NumPy video arrays. The analysis operates at two levels simultaneously: at the per-video level, it extracts spatial, temporal, quality, motion, and colour statistics from sampled frames; at the dataset level, it aggregates these statistics across all videos to reveal distributions, anomalies, and dataset-wide patterns.

Unlike image EDA, which treats each image independently, VideoEDA understands the temporal dimension. It tracks how brightness, contrast, and sharpness evolve across a video's duration, detects moments of high motion and motion blur, identifies scene transitions, and quantifies the lighting stability of the entire recording.

## 1.2 Why Video EDA Matters

Before training any video understanding model — action recognition, activity detection, anomaly detection, or video captioning — understanding the dataset is not optional. The following questions should be answered before training begins:

- What is the distribution of video durations and frame counts? Very short or very long videos may need to be excluded or segmented.
- What fraction of frames are blurry? Motion blur and defocus blur degrade training signal differently and should be quantified separately.
- Are there scene changes? A video with many scene cuts is structurally different from one with a continuous scene, and this affects temporal modelling assumptions.
- Is the lighting stable? Flicker and inconsistent illumination introduce spurious temporal signals that confuse models.
- How diverse are the videos in the dataset? Pairwise distance analysis reveals clusters of similar videos that may cause imbalanced training.
- Are there corrupt or unreadable files that would cause training to crash silently?

VideoEDA answers all of these questions automatically, producing both numeric summaries and visualisations.

## 1.3 Design Philosophy

VideoEDA follows the same four principles as the rest of the VisEDA library:

- Zero required configuration. Point VideoEDA at a directory and it works. Every parameter has a sensible default.
- Progressive disclosure. `summary()` gives you the numbers; `plot_dataset()` gives you the pictures; `report()` gives you a shareable HTML document.
- Memory safety. Videos are not loaded entirely into RAM. VideoEDA samples frames at a configurable rate (`frame_sample_rate`) and enforces a per-video frame cap (`max_frames_per_video`), so even large datasets of long videos are processed safely.
- Lazy OpenCV import. OpenCV is only imported when processing video files. NumPy array inputs bypass OpenCV entirely, making VideoEDA usable in environments where OpenCV is difficult to install.

## 1.4 Installation

Install the core library:

```
pip install viseda
```

VideoEDA requires OpenCV for reading video files from disk:

```
pip install opencv-python
```

For headless server environments (no display):

```
pip install opencv-python-headless
```

Note: OpenCV is only required for file-based loading. If you are working with NumPy arrays directly via `load_arrays()`, OpenCV is not needed.

## 1.5 Supported Video Formats

VideoEDA supports the following file extensions via OpenCV's VideoCapture:

| Extension    | Format                 | Notes                                                |
|--------------|------------------------|------------------------------------------------------|
| .mp4         | MPEG-4 Part 14         | Most common; H.264/H.265 codec support depends on OS |
| .avi         | Audio Video Interleave | Microsoft container; widely supported                |
| .mov         | QuickTime Movie        | Apple container; H.264 usually supported             |
| .mkv         | Matroska               | Open container; broad codec support                  |
| .webm        | WebM                   | Web-optimised; VP8/VP9 codecs                        |
| .mpeg / .mpg | MPEG-1/2               | Legacy broadcast formats                             |
| .m4v         | iTunes Video           | Apple variant of MP4                                 |

Note: Codec support is determined by the OpenCV build and operating system. If a video file produces zero frames, the codec may not be installed. Use `ffmpeg -i` to inspect codec information.

## Chapter 2 — Quick Start

### 2.1 Single Video

```
from viseda import VideoEDA

eda = VideoEDA()
eda.load("video.mp4")

# Get statistics
summary = eda.summary()
print(summary["quality"]["sharpness_mean"])
print(summary["motion"]["motion_blur_fraction"])

# Single video dashboard
eda.plot(video_index=0)

# Temporal deep dive
eda.plot_temporal_statistics(video_index=0)

# Motion/sharpness profile
eda.plot_motion_profile(video_index=0)

# Grid of sampled frames
eda.plot_frame_samples(video_index=0, n=12, cols=4)

# HTML report
eda.report("my_video_report.html")
```

### 2.2 Video Dataset from Directory

The most common pattern for ML datasets is a directory where sub-folders represent class labels:

```
# Dataset structure:
# UCF-101/
#   ApplyEyeMakeup/
#     v_ApplyEyeMakeup_g01_c01.avi
#     ...
#   Archery/
#     v_Archery_g01_c01.avi
#     ...

from viseda import VideoEDA

eda = VideoEDA(
    verbose=True,
    frame_sample_rate=5,          # sample every 5th frame
    max_frames_per_video=300,     # cap at 300 frames per video
    blur_threshold=80.0,          # Laplacian threshold for blur
    motion_blur_threshold=80.0,   # sharpness threshold for motion blur
)

eda.load("UCF-101/", label_from_parent=True)

summary = eda.summary()
print(f"Total videos: {summary['inventory']['total_videos']}")
print(f"Label distribution: {summary['inventory']['label_distribution']}")
print(f"Mean duration: {summary['temporal']['duration_sec']['mean']:.2f}s")
print(f"Mean motion blur fraction: {summary['motion']['motion_blur_fraction']['mean']:.4f}")

eda.plot_dataset()               # full dataset dashboard
eda.plot_pairwise_video_distances() # diversity heatmap
eda.plot_videos_grid(n=16)       # thumbnail grid
eda.report("dataset_report.html") # HTML report
```



## 2.3 NumPy Array Videos

Use `load_arrays()` when videos are already in memory as NumPy arrays:

```
import numpy as np
from viseda import VideoEDA

# Each array: (T, H, W, 3) – uint8 [0,255] or float32 [0,1]
# T = frames, H = height, W = width
videos = [
    np.random.randint(0, 255, (60, 96, 128, 3), dtype=np.uint8),
    np.random.randint(0, 255, (90, 96, 128, 3), dtype=np.uint8),
]
labels = ["traffic", "sports"]

eda = VideoEDA(verbose=True, frame_sample_rate=3)
eda.load_arrays(videos, labels=labels, fps=30.0)

summary = eda.summary()
eda.plot_dataset()
```

## 2.4 CLI Usage

VideoEDA is also accessible via the `viseda` command-line tool:

```
# Analyse a dataset directory
viseda video path/to/dataset/ --label-from-parent

# Save an HTML report
viseda video path/to/dataset/ --report report.html

# Show the dataset dashboard
viseda video path/to/dataset/ --plot

# Show a single video's dashboard
viseda video path/to/video.mp4 --plot --single-plot

# Fast analysis: sample every 10th frame, cap at 100 frames
viseda video path/to/dataset/ \
    --frame-sample-rate 10 \
    --max-frames-per-video 100

# Full options
viseda video path/to/dataset/ \
    --frame-sample-rate 5 \
    --max-frames-per-video 300 \
    --blur-threshold 80.0 \
    --motion-blur-threshold 80.0 \
    --label-from-parent \
    --report report.html \
    --plot \
    --save-plot dashboard.png
```

## Chapter 3 — The VideoEDA Class

### 3.1 Constructor Parameters

All configuration is set at construction time:

```
from viseda import VideoEDA

eda = VideoEDA(
    verbose           = True,      # print progress messages
    max_videos        = None,      # analyse all videos (or set a limit)
    frame_sample_rate = 5,         # sample every Nth frame
    max_frames_per_video = 300,    # cap sampled frames per video
    blur_threshold     = 80.0,     # Laplacian var below = blurry frame
    motion_blur_threshold = 80.0,  # sharpness threshold for motion blur
    scene_change_threshold = 35.0, # frame diff threshold for scene cuts
    resize_width       = 320,      # resize frames to this width (speed)
)
```

| Parameter              | Type       | Default | Description                                                                                                                                                                                            |
|------------------------|------------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| verbose                | bool       | True    | Print progress messages during loading. Shows per-video progress.                                                                                                                                      |
| max_videos             | int   None | None    | Cap the number of videos analysed. Useful for quick exploratory runs.                                                                                                                                  |
| frame_sample_rate      | int        | 5       | Sample every Nth frame. A value of 5 samples 20% of frames. Higher = faster but less accurate temporal statistics.                                                                                     |
| max_frames_per_video   | int        | 300     | Maximum sampled frames per video. Prevents memory issues with very long videos. At 30 fps with rate=5, this covers 50 seconds of video.                                                                |
| blur_threshold         | float      | 80.0    | Laplacian variance below this value flags a frame as blurry. The blur_fraction metric counts the fraction of blurry frames.                                                                            |
| motion_blur_threshold  | float      | 80.0    | Sharpness threshold used together with high frame difference to detect motion-blurred frames. A frame is motion-blurred if its frame diff is in the top 25% AND its sharpness is below this threshold. |
| scene_change_threshold | float      | 35.0    | Mean absolute pixel difference between consecutive frames above this value triggers a scene change count. Typical natural motion is 5-20; cuts are usually 30-100.                                     |
| resize_width           | int   None | 320     | Resize frames to this width for processing. Reduces memory and speeds up computation significantly. None = no resize (use original resolution).                                                        |

### 3.2 load() — File Loading

```
eda.load(
    source          = "path/to/dataset/",
    labels          = None,      # optional {path: label} dict
    label_from_parent = False,   # infer label from parent folder name
)
```

```

    recursive      = True,      # recurse into sub-directories
)

```

The `load()` method walks the source path, discovers supported video files, reads each video using OpenCV, samples frames at the configured rate, and computes all statistics. Corrupt or unreadable files are marked with `is_corrupt=True` and their error message is stored in `record.error`.

Usage patterns:

```

# Pattern 1: class sub-folders (most common for ML datasets)
eda.load("UCF-101/", label_from_parent=True)

# Pattern 2: flat directory, no labels
eda.load("raw_videos/")

# Pattern 3: manual label mapping
labels = {"clip001.mp4": "running", "clip002.mp4": "jumping"}
eda.load("videos/", labels=labels)

# Pattern 4: explicit list of paths
from pathlib import Path
paths = list(Path("videos").glob("**/*.mp4"))[:100]
eda.load(paths)

# Pattern 5: single file
eda.load("video.mp4")

```

### 3.3 `load_arrays()` — Array Loading

```

eda.load_arrays(
    videos = [...],      # list of (T, H, W, 3) arrays
    labels = None,        # optional list of label strings
    names  = None,        # optional list of name strings
    fps    = 30.0,        # scalar or list of per-video fps values
)

```

Accepts float32 in [0,1] or uint8 in [0,255]. Single-channel arrays are broadcast to 3-channel. Arrays containing NaN or infinite values are marked corrupt.

Note: `load_arrays()` does not require OpenCV. The `_cv2()` import is skipped entirely when using array inputs.

### 3.4 `summary()` — Statistics Dictionary

Returns a nested dictionary with six top-level sections:

```

s = eda.summary()

# s["inventory"] — counts, formats, label distribution
# s["spatial"]  — height, width, aspect ratio, megapixels
# s["temporal"] — frame count, fps, duration, temporal consistency
# s["quality"]  — brightness, contrast, sharpness, blur fraction
# s["motion"]   — frame diff, motion intensity, motion blur, scene changes
# s["colour"]   — per-channel RGB mean and std
# s["labels"]   — label distribution, class imbalance ratio

```

Every numeric field is a statistics dict with the same structure:

```

# Example: s["temporal"]["duration_sec"]
{
    "count": 373,
    "mean": 6.63,

```

```

    "std": 3.12,
    "min": 1.00,
    "p25": 4.00,
    "median": 6.00,
    "p75": 8.00,
    "max": 22.60
}

```

### 3.5 get\_record() and get\_frames()

```

# Access the VideoRecord for a specific video
rec = eda.get_record(index=0)
print(rec.path, rec.n_frames, rec.sharpness_mean)

# Access the sampled frames array (T, H, W, 3) uint8
frames = eda.get_frames(index=0)
print(frames.shape) # e.g. (60, 96, 128, 3)

# Iterate over all records
for rec in eda._records:
    if not rec.is_corrupt:
        print(rec.path, rec.blur_fraction)

```

### 3.6 temporal\_profile()

Returns per-frame statistics for a single video — the raw time series behind the temporal plots. Useful for custom analysis beyond the built-in visualisations.

```

profile = eda.temporal_profile(index=0)

# Keys:
# profile["brightness"] - ndarray shape (T,), mean luminance per frame
# profile["contrast"]   - ndarray shape (T,), std of luminance per frame
# profile["sharpness"]  - ndarray shape (T,), Laplacian variance per frame
# profile["frame_diff"] - ndarray shape (T-1,), mean abs diff to prev frame

import matplotlib.pyplot as plt
plt.plot(profile["brightness"], label="Brightness")
plt.plot(profile["sharpness"], label="Sharpness")
plt.legend()
plt.title("Temporal Profile")
plt.show()

```

### 3.7 pairwise\_video\_distances()

Computes a pairwise Euclidean distance matrix between all loaded videos in a normalised feature space. The feature vector used for each video includes: n\_frames, fps, duration\_sec, height, width, brightness\_mean, contrast\_mean, sharpness\_mean, blur\_fraction, motion\_blur\_fraction, motion\_intensity\_mean, scene\_change\_rate. All features are z-score normalised before distance computation, so no single feature dominates.

```

D, names = eda.pairwise_video_distances(max_videos=50)

# D is a symmetric matrix shape (N, N) of Euclidean distances
# names is a list of N label/stem strings

# Find the most similar pair
import numpy as np
np.fill_diagonal(D, np.inf)
i, j = np.unravel_index(D.argmin(), D.shape)
print(f"Most similar: {names[i]} and {names[j]} (distance={D[i,j]:.3f})")

```

## 3.8 report()

```
# Generate a self-contained HTML report  
eda.report("viseda_video_report.html")
```

The HTML report includes all `summary()` sections formatted in a card layout. No internet connection is required to view it. The file is safe to share via email or upload to a server.

## Chapter 4 — The VideoRecord Object

Every analysed video produces one VideoRecord object. These are stored in `eda._records` and are accessible via `eda.get_record(index)`. The record is a lightweight container for all per-video statistics computed during loading.

### 4.1 Identity Fields

| Field        | Type       | Description                                      |
|--------------|------------|--------------------------------------------------|
| path         | str        | File path or " for array-loaded videos           |
| label        | str None   | Class label (from folder name or label dict)     |
| file_ext     | str        | File extension e.g. '.mp4', '.avi', or 'array'   |
| file_size_kb | float None | File size in kilobytes (None for array inputs)   |
| is_corrupt   | bool       | True if the video could not be read or processed |
| error        | str None   | Error message if <code>is_corrupt</code> is True |

### 4.2 Spatial Fields

| Field        | Type  | Description                                              |
|--------------|-------|----------------------------------------------------------|
| height       | int   | Frame height in pixels (after optional resize)           |
| width        | int   | Frame width in pixels (after optional resize)            |
| channels     | int   | Number of colour channels (always 3 after normalisation) |
| aspect_ratio | float | width/height                                             |
| megapixels   | float | height × width / 1,000,000                               |

### 4.3 Temporal Fields

| Field                   | Type  | Description                                                                                    |
|-------------------------|-------|------------------------------------------------------------------------------------------------|
| n_frames                | int   | Total frame count reported by the video container                                              |
| sampled_frames          | int   | Number of frames actually sampled and analysed                                                 |
| fps                     | float | Frames per second reported by the container                                                    |
| duration_sec            | float | $n\_frames / fps$ — total video duration                                                       |
| temporal_brightness_std | float | Std of per-frame mean brightness across sampled frames. High = flickering or lighting changes. |
| temporal_contrast_std   | float | Std of per-frame contrast across sampled frames.                                               |

### 4.4 Quality Fields

| Field           | Type  | Description                                                |
|-----------------|-------|------------------------------------------------------------|
| brightness_mean | float | Mean of per-frame mean luminance across all sampled frames |
| brightness_std  | float | Std of per-frame mean luminance (inter-frame variation)    |

|                |       |                                                            |
|----------------|-------|------------------------------------------------------------|
| contrast_mean  | float | Mean of per-frame contrast (std of pixel luminance)        |
| contrast_std   | float | Std of per-frame contrast across frames                    |
| sharpness_mean | float | Mean Laplacian variance across all sampled frames          |
| sharpness_std  | float | Std of per-frame sharpness                                 |
| sharpness_min  | float | Minimum per-frame sharpness (worst frame in the video)     |
| blur_fraction  | float | Fraction of sampled frames with sharpness < blur_threshold |

## 4.5 Motion Fields

| Field                 | Type  | Description                                                                                                          |
|-----------------------|-------|----------------------------------------------------------------------------------------------------------------------|
| frame_diff_mean       | float | Mean absolute pixel difference between consecutive frames. Primary motion intensity measure.                         |
| frame_diff_std        | float | Std of frame differences. High = inconsistent motion.                                                                |
| motion_intensity_mean | float | Same as frame_diff_mean (retained for API clarity)                                                                   |
| motion_intensity_std  | float | Same as frame_diff_std                                                                                               |
| motion_blur_fraction  | float | Fraction of frames that are simultaneously high-motion (diff in top 25%) AND low-sharpness (< motion_blur_threshold) |
| scene_change_count    | int   | Number of frame transitions where diff > scene_change_threshold                                                      |
| scene_change_rate     | float | scene_change_count / (sampled_frames - 1)                                                                            |

## 4.6 Colour Fields

| Field         | Type         | Description                                                                                           |
|---------------|--------------|-------------------------------------------------------------------------------------------------------|
| rgb_mean      | ndarray (3,) | Per-channel mean pixel values across all sampled frames, normalised to [0,1]. Order: R, G, B.         |
| rgb_std       | ndarray (3,) | Per-channel std of pixel values across all sampled frames.                                            |
| sample_frames | ndarray      | Up to 6 evenly-spaced sampled frames for thumbnail previews. Shape: (min(T,6), H, W, 3), dtype uint8. |

## Chapter 5 — What Is Analysed

### 5.1 Frame Sampling Strategy

VideoEDA does not analyse every frame. Doing so would be prohibitively slow for large datasets — a 10-minute video at 30 fps contains 18,000 frames. Instead, frames are sampled at a fixed interval controlled by `frame_sample_rate` (default: every 5th frame) and capped at `max_frames_per_video` (default: 300 frames).

The sampling starts from frame 0 and takes every Nth frame until the cap is reached or the video ends. This gives uniform temporal coverage across the video. For a 60-second video at 30 fps with `frame_sample_rate=5` and `max_frames_per_video=300`, all frames are sampled ( $60s \times 30fps / 5 = 360$  but capped at 300, covering ~50 seconds).

For array inputs via `load_arrays()`, the same logic applies: frames at indices 0, N, 2N, ... are selected up to the configured cap.

Note: Increasing `frame_sample_rate` reduces analysis time but makes temporal statistics coarser. For motion analysis, a rate of 1-3 is recommended. For quality statistics like brightness and sharpness, a rate of 5-10 is usually sufficient.

### 5.2 Spatial Properties

Spatial properties are measured once per video from the first sampled frame:

- **Height and Width:** Frame dimensions in pixels, measured after optional resizing. If `resize_width` is set, frames are resized proportionally.
- **Aspect Ratio:** Width divided by height. Values above 1.0 are landscape; below 1.0 are portrait; 1.0 is square. The standard 16:9 widescreen ratio is 1.778.
- **Megapixels:** Height  $\times$  width / 1,000,000. A 320 $\times$ 180 frame is 0.0576 MP.

At the dataset level, the spatial distribution reveals resolution inconsistencies. The resolution scatter plot in `plot_dataset()` shows each video as a point at (width, height), making it easy to spot mixed-resolution datasets.

### 5.3 Brightness

Brightness is computed for each sampled frame as the mean luminance using the ITU-R BT.601 formula:

$$L = 0.299 \times R + 0.587 \times G + 0.114 \times B$$

The weights reflect human visual sensitivity: the eye is most sensitive to green, least to blue. Range is 0 (pure black) to 255 (pure white).

Two brightness statistics are stored per video:

- **brightness\_mean:** Average brightness across all sampled frames. A consistently dark video will have a low value regardless of frame-to-frame variation.
- **temporal\_brightness\_std:** Standard deviation of per-frame brightness across the video. This is the key temporal consistency metric. A stable video has low `temporal_brightness_std`; a flickering video has high values.

| temporal_brightness_std | Interpretation                                                          |
|-------------------------|-------------------------------------------------------------------------|
| < 1.0                   | Very stable lighting — ideal for most applications                      |
| 1–5                     | Normal variation — typical of moving subjects or slight lighting change |
| 5–15                    | Noticeable flicker or lighting transitions                              |
| > 15                    | Strong flicker, rapid lighting changes, or strobe effects               |



## 5.4 Contrast

Contrast is computed per frame as the standard deviation of pixel luminance values across the frame. A frame where all pixels have similar brightness has low contrast (near 0); a frame with bright highlights and dark shadows has high contrast.

contrast\_mean is the average across sampled frames. Low contrast\_mean may indicate foggy, hazy, or poorly lit footage. For a typical outdoor sports dataset, values between 30–70 are expected.

## 5.5 Sharpness and Blur Detection

Sharpness is computed per frame using the variance of the Laplacian operator:

```
sharpness = Var(Laplacian(frame))
```

The Laplacian is a second-derivative edge detector. Sharp frames have many strong edges and produce high Laplacian variance. Blurry frames have few edges and produce low variance. This measure is robust to image brightness and content.

Three sharpness statistics are stored:

- sharpness\_mean: Mean sharpness across all sampled frames. A sharp, clear video like a broadcast news programme might score 500–2000. A hand-held footage video might score 50–300.
- sharpness\_std: Variation in sharpness across frames. High std indicates that some frames are sharp and some are blurry — typical of action footage.
- sharpness\_min: The sharpness of the worst (most blurry) frame. Useful for detecting videos that contain at least one severely blurry frame.

Blur Detection (blur\_fraction): A frame is flagged as blurry if its Laplacian variance is below blur\_threshold (default 80). The blur\_fraction is the proportion of sampled frames that are blurry.

| blur_fraction | Interpretation               | Recommended Action                       |
|---------------|------------------------------|------------------------------------------|
| 0.00          | No blurry frames             | No action needed                         |
| 0.01–0.05     | Minor blur (~1–5% of frames) | Review; likely acceptable                |
| 0.05–0.20     | Moderate blur                | Investigate; may affect training         |
| 0.20–0.50     | Severe blur                  | Consider exclusion or filtering          |
| > 0.50        | Majority of frames blurry    | Likely a defocused or motion-heavy video |

## 5.6 Noise and Compression

VideoEDA does not currently compute a standalone noise estimate metric (unlike ImageEDA, which subtracts a Gaussian-blurred version). However, sensor noise is implicitly captured in the frame\_diff\_mean when motion is low: a static scene with high frame difference indicates sensor noise or compression artefacts between frames.

Note: A future release will add explicit noise and JPEG blockiness estimation for compressed video frames.

## 5.7 Frame Difference and Motion Intensity

Frame difference is the primary motion measure in VideoEDA. It is computed as the mean absolute pixel difference between consecutive sampled frames:

```
frame_diff[t] = mean(|frame[t] - frame[t-1]|)
```

This produces an array of length T-1 (one value per consecutive pair). Two aggregate statistics are computed:

- `frame_diff_mean` / `motion_intensity_mean`: Mean across all consecutive pairs. A low value means the video is static or slow-moving; a high value means fast or frequent motion.
- `frame_diff_std` / `motion_intensity_std`: Std across pairs. Low std means consistently smooth or consistently fast motion; high std means irregular motion — periods of stillness alternating with bursts of movement.

| <code>motion_intensity_mean</code> | Interpretation                                           |
|------------------------------------|----------------------------------------------------------|
| < 2                                | Nearly static — surveillance footage, fixed camera       |
| 2–10                               | Slow motion — walking, gentle panning                    |
| 10–25                              | Moderate motion — normal activity, typical action videos |
| > 25                               | Fast motion — sports, rapid camera movements, scene cuts |

## 5.8 Motion Blur Detection

Motion blur is a specific combination of high motion and low sharpness in the same frame. A frame is classified as motion-blurred when both conditions hold simultaneously:

```
motion_blurred = (frame_diff > 75th_percentile_of_diffs)
                  AND (sharpness < motion_blur_threshold)
```

The 75th percentile threshold adapts to the video's own motion range, so a static video with minor sensor jitter is not flagged. The `motion_blur_fraction` is the proportion of consecutive frame pairs satisfying both conditions.

This metric is uniquely valuable for datasets used in action recognition or autonomous driving, where motion blur is a systematic data quality issue rather than an occasional artefact.

Note: `motion_blur_fraction` = 0.0 for videos with only one sampled frame, since no frame differences can be computed.

## 5.9 Scene Change Detection

A scene change (cut) is detected when the frame difference exceeds `scene_change_threshold` (default: 35.0). Two statistics are computed:

- `scene_change_count`: Number of detected scene cuts in the video.
- `scene_change_rate`: `scene_change_count` / (`sampled_frames` - 1). Normalises by the number of frame transitions to allow comparison across videos of different lengths.

Scene change detection is relevant for:

- Action recognition: Videos with many cuts may have unreliable temporal context for models that assume temporal continuity.
- Surveillance: Continuous footage should have zero scene changes. Non-zero counts may indicate recording interruptions.
- Editing analysis: The scene change rate directly measures editing pace.

Note: The threshold of 35.0 works well for typical camera footage. For highly compressed videos, lower the threshold. For fast-motion sports, increase it to avoid false positives from rapid but continuous motion.

## 5.10 Temporal Consistency

Temporal consistency measures how stable a video's visual properties are across its duration. Two metrics are computed:

- `temporal_brightness_std`: Standard deviation of per-frame mean brightness. Measures lighting stability. Low = stable illumination; high = flicker, lighting changes, or day/night transitions.
- `temporal_contrast_std`: Standard deviation of per-frame contrast. Measures whether the visual complexity of scenes changes over time.

These metrics are especially important for:

- Surveillance systems: High temporal\_brightness\_std may indicate automatic exposure adjustment (AGC) or IR cut-filter switching.
- Indoor sports: Fluorescent lighting flicker at 50/60 Hz can produce sinusoidal brightness oscillations detectable with high temporal\_brightness\_std.
- Autonomous driving: Driving from tunnel to open road produces a sharp brightness transition visible as high temporal\_brightness\_std.

## 5.11 Colour Analysis

Colour statistics are computed by averaging across all pixels in all sampled frames:

- rgb\_mean: Per-channel (R, G, B) mean pixel value in [0, 1]. Reveals colour cast: a dataset with consistently high R and low B is warm-toned.
- rgb\_std: Per-channel std of pixel values across all sampled frames. Low std means the video is colour-consistent; high std means significant colour variation.

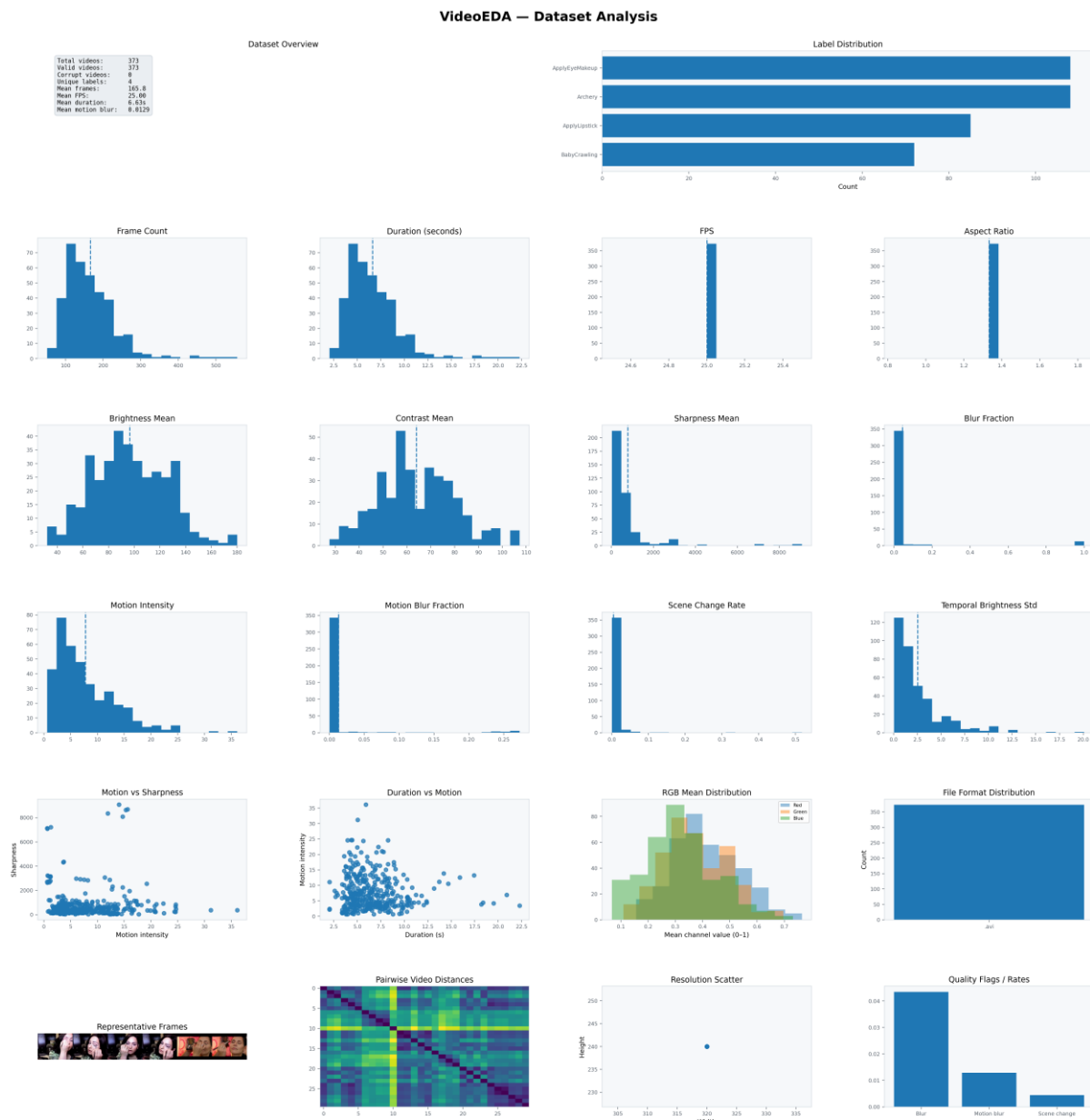
The RGB Mean Distribution plot in plot\_dataset() overlays the R, G, B histograms across all videos, revealing systematic colour biases in the dataset.

## Chapter 6 — Visualisations

### 6.1 plot\_dataset() — Dataset Dashboard

The dataset dashboard as shown in Figure 6-1 is a six-row, four-column grid that provides a complete overview of the entire video dataset in one figure. It is the primary starting point for any new dataset and is equivalent to the main dashboard in ImageEDA. Call it as:

```
eda.plot_dataset(
    figsize = (24, 24),          # figure size in inches
    save_path = "dashboard.png", # or None to show interactively
    dpi      = 150,
)
```



**Figure 6.1 — VideoEDA Dataset Dashboard**

*The complete dataset-level dashboard. It combines inventory, temporal, quality, motion, colour, representative-frame and diversity analyses in a single figure.*

## Row 0: Dataset Overview and Label Distribution

Dataset Overview card (columns 0–1): A monospaced text card showing the eight most important dataset facts at a glance:

- Total, valid, and corrupt video counts
- Number of unique class labels
- Mean frame count across all videos
- Mean FPS across all videos
- Mean duration in seconds
- Mean motion blur fraction — a key quality indicator

Label Distribution (columns 2–3): A horizontal bar chart showing how many videos belong to each class label. Class labels are sorted by count descending, showing up to 20 classes. This is the first diagnostic to check — a class imbalance ratio above 5:1 is a red flag for ML training.

## Row 1: Spatial and Temporal Inventory

Four histograms characterising the temporal structure of the dataset:

- Frame Count: Distribution of total frame counts. The dashed vertical line marks the mean. A wide distribution means the dataset has very short and very long videos — which requires handling in temporal models (padding/truncation).
- Duration (seconds): Distribution of video durations. For a typical action recognition dataset like UCF-101, this peaks at 4–10 seconds.
- FPS: Frames-per-second distribution. Most cameras record at 24, 25, or 30 fps. A spike at a single value (e.g., 25.0) indicates a homogeneous dataset.
- Aspect Ratio: Distribution of width/height ratios. A spike at 1.78 means all videos are 16:9. A wide distribution indicates mixed sources.

## Row 2: Quality Metrics

Four histograms characterising per-video quality:

- Brightness Mean: Distribution of average brightness across videos. A bimodal distribution may indicate two distinct scene types (indoor/outdoor, day/night).
- Contrast Mean: Distribution of average contrast. Low-contrast videos from foggy or uniformly-lit scenes will cluster at the low end.
- Sharpness Mean: Distribution of average Laplacian variance. Right-skewed distribution is typical — most videos are moderately sharp, a small fraction are very sharp.
- Blur Fraction: Distribution of the proportion of blurry frames. Ideally a spike near 0 for most videos, with a long tail. Videos with `blur_fraction > 0.5` should be reviewed.

## Row 3: Motion and Temporal Consistency

- Motion Intensity: Distribution of mean frame difference. Action videos cluster at higher values; surveillance footage clusters near zero.
- Motion Blur Fraction: Distribution of the motion-blur metric. Most videos should have near-zero values. Consistently high values indicate fast-motion or poorly-exposed footage.
- Scene Change Rate: Distribution of cut frequency. Continuous footage has zero; edited content has non-zero rates.
- Temporal Brightness Std: Distribution of lighting stability. Values above 5 indicate significant flickering or lighting transitions.

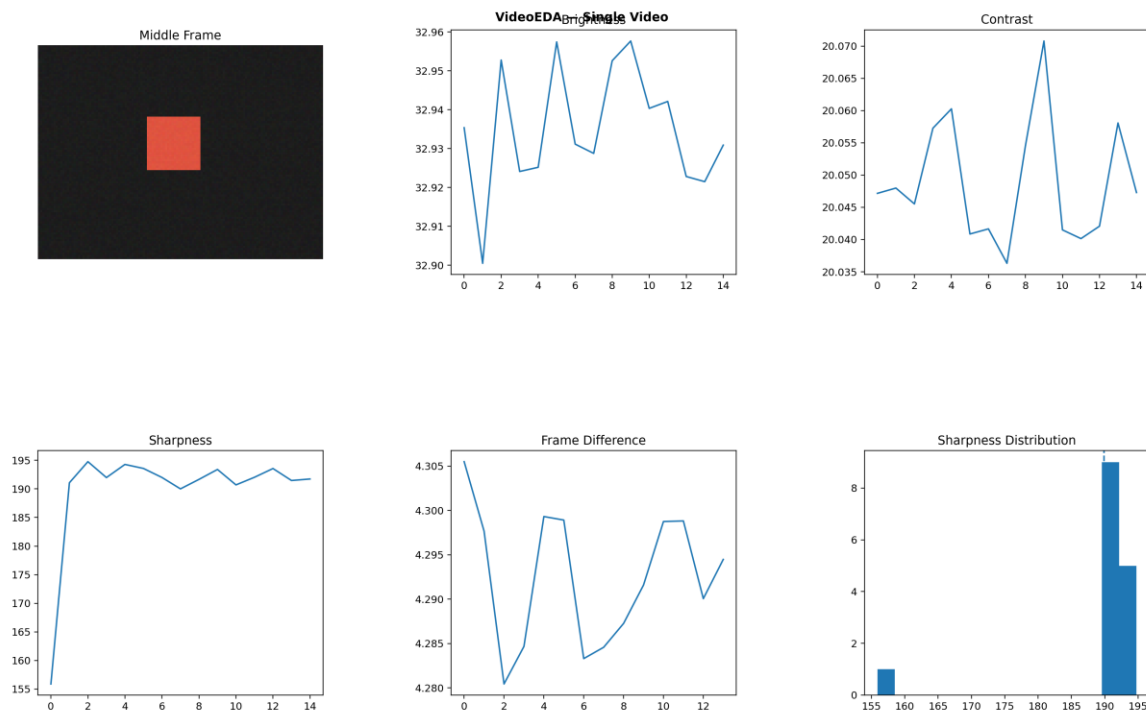
## Row 4: Relationships and Colour

- Motion vs Sharpness scatter: Each video is a point at (motion\_intensity\_mean, sharpness\_mean). The inverse relationship — high motion tends to correlate with low sharpness — is the signature of motion blur. Videos in the high-motion, high-sharpness quadrant are well-exposed fast-motion videos; those in high-motion, low-sharpness are motion-blurred.
- Duration vs Motion scatter: Each video at (duration\_sec, motion\_intensity\_mean). Reveals whether longer videos tend to be more or less active. No correlation is typical for well-curated datasets.
- RGB Mean Distribution: Overlapping histograms of per-video mean R, G, and B channel values. A rightward shift of the Red histogram vs Blue confirms a warm colour bias, as seen in outdoor or skin-tone-dominant datasets.
- File Format Distribution: Bar chart showing how many videos are in each format. A purely homogeneous format dataset (.avi only, or .mp4 only) is ideal.

## Row 5: Previews, Diversity, Resolution, Quality Flags

- Representative Frames: A horizontal strip of middle frames from the first six valid videos. Provides a quick visual sanity check — confirming that the videos contain what is expected.
- Pairwise Video Distances: A heatmap of Euclidean distances between all videos in the normalised feature space (up to 30 videos). Dark cells = similar videos; bright/yellow cells = dissimilar videos. Clusters of dark cells reveal groups of nearly identical videos that may indicate dataset redundancy.
- Resolution Scatter: Each video as a point at (width, height). A tight cluster indicates a homogeneous dataset; spread points indicate mixed resolutions requiring normalisation.
- Quality Flags / Rates: A bar chart showing three mean rates across the dataset: blur fraction, motion blur fraction, and scene change rate. These three metrics summarise the overall quality at a glance.

## 6.2 plot() — Single Video Dashboard



**Figure 6.2 — Single Video Dashboard**

*The single-video dashboard showing a representative frame and temporal brightness, contrast, sharpness and frame-difference behaviour.*

```
eda.plot(
    video_index = 0,
    figsize     = (16, 10),
    save_path   = None,
    dpi         = 150,
)
```

The single video dashboard is a 2×3 grid showing six panels for one video:

| Panel                  | Position     | Content                                                                                        |
|------------------------|--------------|------------------------------------------------------------------------------------------------|
| Middle Frame           | Row 0, Col 0 | The frame at the midpoint of the video. Provides a representative visual preview.              |
| Brightness             | Row 0, Col 1 | Per-frame mean brightness over sampled frames. Shows lighting evolution across the video.      |
| Contrast               | Row 0, Col 2 | Per-frame contrast (luminance std) over frames.                                                |
| Sharpness              | Row 1, Col 0 | Per-frame Laplacian variance. Dips indicate blurry sections; peaks indicate sharp sections.    |
| Frame Difference       | Row 1, Col 1 | Motion signal: mean abs pixel diff between consecutive frames. Spikes = motion bursts or cuts. |
| Sharpness Distribution | Row 1, Col 2 | Histogram of per-frame sharpness values. The dashed line marks the blur threshold.             |

Interpretation: In the single video dashboard from the UCF-101 dataset, the brightness panel shows a gradual increase (subject moving from shadow to light), the frame difference panel spikes at a scene transition, and the sharpness distribution is bimodal — most frames are sharp (right peak) with a smaller cluster of motion-blurred frames (left peak).

6.3 plot\_motion\_profile() — Motion/Sharpness Profile

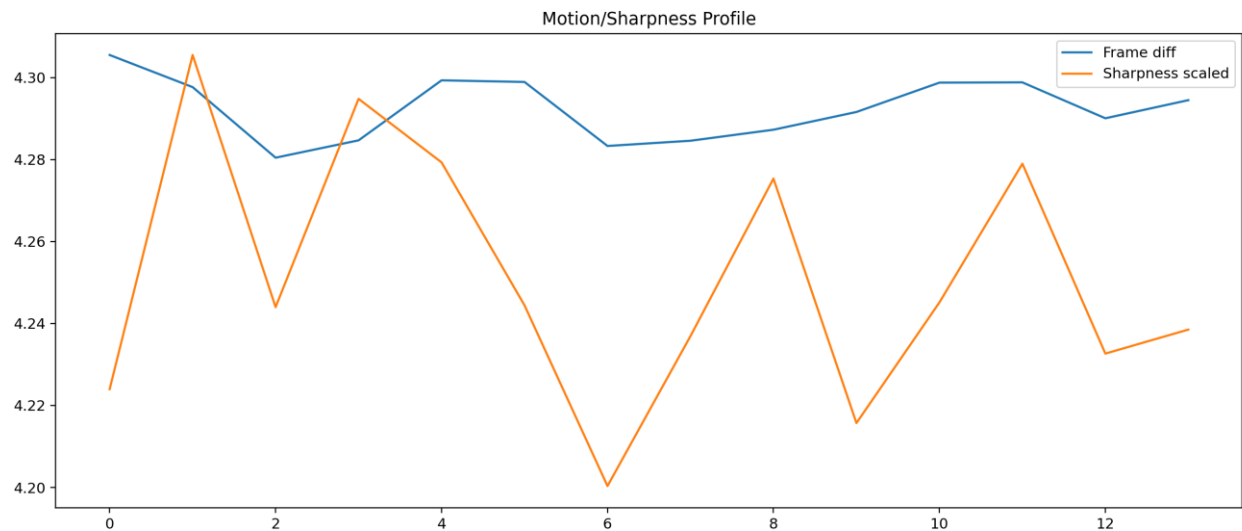


Figure 6.3 — Motion/Sharpness Profile

Frame difference and scaled sharpness plotted together to show whether high-motion sections coincide with sharpness drops.

```
eda.plot_motion_profile(
    video_index = 0,
    save_path   = None,
```

```
    dpi      = 150,
)
```

Plots two overlaid time series for a single video on the same axis:

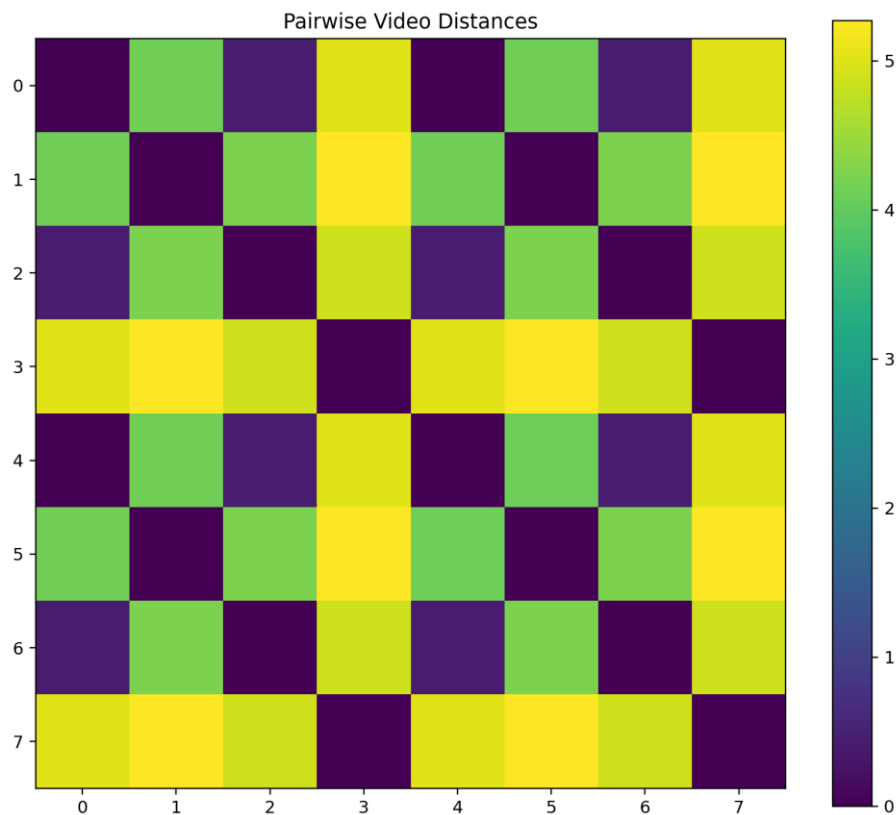
- Frame difference (blue): The raw motion signal — mean absolute pixel difference between consecutive sampled frames.
- Sharpness scaled (orange): Per-frame Laplacian variance, normalised to the same scale as the frame difference for visual comparison.

The key insight from this plot is the inverse correlation between motion and sharpness. When the frame difference spikes (fast motion), the sharpness typically drops (motion blur). When they move in sync, it suggests a scene cut rather than motion blur — cuts reset both signals abruptly.

This plot is particularly useful for:

- Drone footage: identifying when the drone is accelerating vs hovering
- Sports: identifying plays vs stoppage time
- Surveillance: identifying events (motion spikes) in otherwise static footage

## 6.4 `plot_pairwise_video_distances()` — Diversity Heatmap



**Figure 6.4 — Pairwise Video Distance Heatmap**

*A heatmap of normalised feature-space distances between videos. Dark cells represent similarity; brighter cells represent diversity.*

```
eda.plot_pairwise_video_distances(
    save_path = None,
    dpi      = 150,
)
```



Displays the full pairwise distance matrix as a colour heatmap. Each cell (i, j) shows the Euclidean distance between videos i and j in the normalised 12-dimensional feature space. The diagonal is always zero (a video is identical to itself).

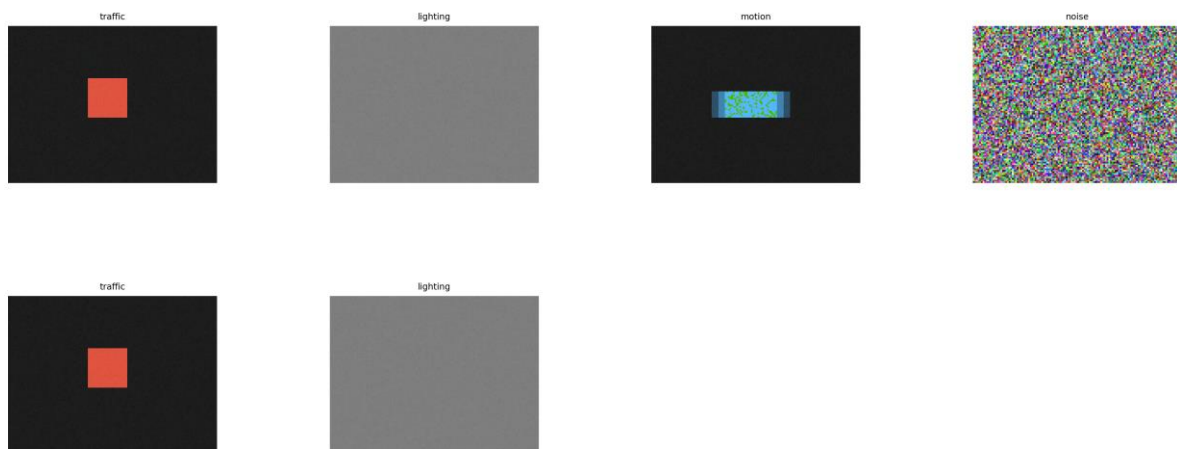
Reading the heatmap:

- Dark cells (distance  $\approx 0$ ): Nearly identical videos. In the test output from synthetic data with alternating scenes, the checkerboard pattern shows that every other video is similar — videos of the same scene type cluster together.
- Yellow cells (high distance): Very different videos. High distance between all pairs indicates a diverse dataset — good for training.
- Block structure: Clusters of similar videos. In a well-organised dataset, videos of the same class should form blocks of low distance.

This analysis is unique to VideoEDA among common EDA tools. It enables:

- Redundancy detection: Identifying videos so similar they add no new information to the training set.
- Active learning: Selecting the most diverse subset of videos for annotation by choosing the minimum-distance spanning set.
- Train/val split validation: Ensuring that train and val splits are not dominated by similar videos.

## 6.5 `plot_videos_grid()` — Video Thumbnail Grid



**Figure 6.5 — Video Thumbnail Grid**

*A grid of representative middle frames from sampled videos, useful for visual sanity checking.*

```
eda.plot_videos_grid(
    n        = 12,    # max videos to display
    cols     = 4,     # columns in grid
    save_path = None,
    dpi      = 150,
)
```

Displays a grid of middle frames from up to n videos. Each thumbnail is labelled with the video's class label or filename stem. This is the visual equivalent of `plot_samples()` in ImageEDA — the first sanity check that the dataset actually contains what is expected.

For the UCF-101 dataset, this reveals the visual diversity of different action classes: ApplyEyeMakeup (close-up face, bright lighting), Archery (outdoor, green background), BabyCrawling (floor-level, soft lighting).

## 6.6 `plot_temporal_statistics()` — Temporal Deep Dive

```
eda.plot_temporal_statistics(  
    video_index = 0,  
    save_path   = None,  
    dpi         = 150,  
)
```

A 2×2 detailed temporal analysis for one video, showing brightness, contrast, sharpness, and frame difference each in their own panel with proper axis labels and full resolution. Compared to `plot()`, this provides larger, higher-resolution temporal views for detailed inspection of a single video.

## 6.7 `plot_quality_summary()` — Quality Summary

```
eda.plot_quality_summary(  
    save_path = None,  
    dpi       = 150,  
)
```

A 2×3 quality-focused dashboard for the dataset showing: brightness mean distribution, contrast mean distribution, sharpness mean distribution, blur fraction distribution, motion blur fraction distribution, and a motion vs sharpness scatter. More focused than `plot_dataset()` when quality is the primary concern.

## 6.8 `plot_frame_samples()` — Frame Sample Grid

```
eda.plot_frame_samples(  
    video_index = 0,  
    n           = 12,    # number of frames to show  
    cols        = 4,  
    save_path   = None,  
    dpi         = 150,  
)
```

Displays `n` evenly-spaced sampled frames from a single video in a grid layout. Each frame is labelled with its index in the sampled frame array. This is the most direct way to inspect what VideoEDA actually saw when analysing a video — confirming that the `frame_sample_rate` and `resize_width` settings produced reasonable results.

## Chapter 7 — Interpreting Results and Common Findings

### 7.1 Red Flags to Watch For

The following findings should prompt action before training:

| Finding                                    | Threshold                           | Recommended Action                                                                              |
|--------------------------------------------|-------------------------------------|-------------------------------------------------------------------------------------------------|
| blur_fraction mean > 0.20                  | > 20% blurry frames on average      | Review the blurriest videos. Consider excluding videos with blur_fraction > 0.5 from training.  |
| motion_blur_fraction mean > 0.10           | > 10% motion-blurred frames         | Increase blur_threshold or motion_blur_threshold, or use data augmentation for blur robustness. |
| temporal_brightness_std mean > 5           | High lighting instability           | Investigate the source. Flicker may require video stabilisation preprocessing.                  |
| scene_change_rate mean > 0.20              | Many scene cuts                     | Consider whether multi-shot videos suit the task. Some models require continuous-scene input.   |
| Class imbalance ratio > 5                  | One class is 5× larger than another | Oversample minority classes, or use a class-weighted loss function.                             |
| Corrupt video count > 1%                   | More than 1% unreadable files       | Investigate the source. Re-download or re-encode the corrupt files.                             |
| Pairwise heatmap shows large dark clusters | Many near-identical videos          | Remove duplicates or near-duplicates before creating train/val splits.                          |
| Resolution scatter shows widespread        | Mixed resolutions                   | Standardise all videos to the same resolution in preprocessing.                                 |

### 7.2 Common Dataset Patterns

UCF-101 (Action Recognition): In the dashboard shown in this documentation, 373 videos from 4 classes were analysed. Key findings:

- Mean duration 6.6s, mean FPS 25 — consistent short clips suitable for temporal modelling with fixed-length inputs.
- Mean motion intensity ~5–15 — typical for human activity videos.
- Low motion blur fraction (~0.013) — well-shot, mostly sharp footage.
- Scene change rate near zero — clips are single continuous shots.
- Temporal brightness std ~2–5 — stable lighting with minor variations.

Surveillance datasets: Typically show:

- Very low motion\_intensity\_mean (< 2) for static camera footage
- scene\_change\_count = 0 for continuous recordings
- High temporal\_brightness\_std if AGC (auto gain control) is active

Dashcam/autonomous driving datasets: Typically show:

- Moderate motion\_intensity\_mean (5–20) for normal driving speed
- Sharp bimodal brightness distribution (day vs night driving)
- Higher motion\_blur\_fraction during rapid lane changes or braking

## Chapter 8 — Advanced Usage

### 8.1 Working with Large Datasets

```
# Strategy 1: limit the number of videos
eda = VideoEDA(max_videos=500)
eda.load("large_dataset/")

# Strategy 2: coarser frame sampling
eda = VideoEDA(frame_sample_rate=10, max_frames_per_video=100)
eda.load("large_dataset/")

# Strategy 3: smaller frame size
eda = VideoEDA(resize_width=160) # 160px wide instead of 320px
eda.load("large_dataset/")

# Strategy 4: combine all three for maximum speed
eda = VideoEDA(
    max_videos      = 1000,
    frame_sample_rate = 10,
    max_frames_per_video = 50,
    resize_width     = 160,
    verbose         = True,
)
eda.load("large_dataset/")
```

Note: On a typical laptop, VideoEDA processes approximately 30–60 videos per minute with the default settings (frame\_sample\_rate=5, max\_frames\_per\_video=300, resize\_width=320). Halving resize\_width roughly quadruples processing speed.

### 8.2 Accessing VideoRecord Objects

```
# Find the 10 most blurry videos
blurry = sorted(
    [r for r in eda._records if not r.is_corrupt],
    key=lambda r: r.blur_fraction or 0,
    reverse=True
)[:10]
for r in blurry:
    print(f"{r.path}: blur={r.blur_fraction:.3f}, "
          f"sharpness={r.sharpness_mean:.1f}")

# Find high-motion videos
action = [
    r for r in eda._records
    if not r.is_corrupt
    and r.motion_intensity_mean is not None
    and r.motion_intensity_mean > 20
]
print(f"{len(action)} high-motion videos")

# Export all stats to a DataFrame
import pandas as pd
data = []
for r in eda._records:
    if not r.is_corrupt:
        data.append({
            "path": r.path,
            "label": r.label,
            "duration": r.duration_sec,
            "fps": r.fps,
            "n_frames": r.n_frames,
            "brightness": r.brightness_mean,
            "contrast": r.contrast_mean,
```

```

        "sharpness":      r.sharpness_mean,
        "blur_fraction":  r.blur_fraction,
        "motion":         r.motion_intensity_mean,
        "motion_blur":    r.motion_blur_fraction,
        "scene_changes":  r.scene_change_count,
        "temporal_bri_std": r.temporal_brightness_std,
    })
df = pd.DataFrame(data)
df.to_csv("video_stats.csv", index=False)

```

### 8.3 Integrating with ML Pipelines

```

from viseda import VideoEDA
import torch
from torch.utils.data import DataLoader

# Step 1: Analyse the dataset
eda = VideoEDA(verbose=True)
eda.load("dataset/train/", label_from_parent=True)
s = eda.summary()

# Step 2: Check for data quality issues before training
print(f"Corrupt videos:      {s['inventory']['corrupt_videos']}")
print(f"Mean blur fraction:  {s['quality']['blur_fraction']['mean']:.3f}")
print(f"Class imbalance:      {s['labels']['class_imbalance_ratio']}")
print(f"Mean motion blur:     {s['motion']['motion_blur_fraction']['mean']:.3f}")

# Step 3: Get per-video stats for sampling weights
blur_fractions = [r.blur_fraction or 0 for r in eda._records]
# Use inverse blur as quality weight: high blur = low weight
weights = [1.0 / (bf + 0.01) for bf in blur_fractions]

# Step 4: Get corrupt paths to exclude
corrupt_paths = s['inventory']['corrupt_paths']
print(f"Exclude from training: {corrupt_paths}")

# Step 5: Get mean RGB for normalisation
rgb_mean = s['colour']['rgb_mean_mean']
rgb_std = s['colour']['rgb_std_mean']
print(f"Dataset mean RGB: {rgb_mean}")
print(f"Dataset std RGB: {rgb_std}")

```

### 8.4 Comparing Two Datasets

```

eda_train = VideoEDA(verbose=False)
eda_val = VideoEDA(verbose=False)

eda_train.load("dataset/train/", label_from_parent=True)
eda_val.load("dataset/val/", label_from_parent=True)

s_tr = eda_train.summary()
s_va = eda_val.summary()

# Compare quality metrics
metrics = [
    ("Brightness mean", "quality", "brightness_mean"),
    ("Contrast mean", "quality", "contrast_mean"),
    ("Sharpness mean", "quality", "sharpness_mean"),
    ("Blur fraction", "quality", "blur_fraction"),
    ("Motion intensity", "motion", "motion_intensity_mean"),
    ("Motion blur frac", "motion", "motion_blur_fraction"),
    ("Scene change rate", "motion", "scene_change_rate"),
    ("Duration", "temporal", "duration_sec"),
]
print(f"{'Metric':10} {'Val':>10} {'Diff%':>8}")

```

```
print("-" * 54)
for label, section, key in metrics:
    tr = s_tr[section][key]["mean"]
    va = s_va[section][key]["mean"]
    if tr and va and tr != 0:
        diff_pct = abs(tr - va) / abs(tr) * 100
        flag = " ← !" if diff_pct > 20 else ""
        print(f"{label:10.3f} {va:>10.3f} {diff_pct:>7.1f}%{flag}")

# Large differences (> 20%) in brightness, contrast, or motion
# indicate train/val distribution mismatch
```

## Chapter 9 — API Reference

Complete reference for all public methods of the VideoEDA class.

### VideoEDA.\_\_init\_\_()

| Parameter              | Type     | Default | Description                         |
|------------------------|----------|---------|-------------------------------------|
| verbose                | bool     | True    | Print progress messages             |
| max_videos             | int None | None    | Max videos to analyse               |
| frame_sample_rate      | int      | 5       | Sample every Nth frame              |
| max_frames_per_video   | int      | 300     | Cap on sampled frames per video     |
| blur_threshold         | float    | 80.0    | Laplacian var below = blurry frame  |
| motion_blur_threshold  | float    | 80.0    | Sharpness threshold for motion blur |
| scene_change_threshold | float    | 35.0    | Frame diff threshold for scene cuts |
| resize_width           | int None | 320     | Resize width for processing speed   |

### VideoEDA.load()

| Parameter         | Type          | Default | Description                         |
|-------------------|---------------|---------|-------------------------------------|
| source            | str Path List | —       | Directory, file, or list of paths   |
| labels            | dict None     | None    | Optional {path: label} mapping      |
| label_from_parent | bool          | False   | Infer label from parent folder name |
| recursive         | bool          | True    | Recurse into sub-directories        |

### VideoEDA.load\_arrays()

| Parameter | Type           | Default | Description                                 |
|-----------|----------------|---------|---------------------------------------------|
| videos    | List[ndarray]  | —       | (T,H,W,3) arrays, uint8 or float32 [0,1]    |
| labels    | List[str] None | None    | Optional label per video                    |
| names     | List[str] None | None    | Optional name per video (shown in plots)    |
| fps       | float List     | 30.0    | Frames per second: scalar or per-video list |

### VideoEDA.summary()

Returns a nested dict. Top-level keys:

| Key       | Contents                                                                                                         |
|-----------|------------------------------------------------------------------------------------------------------------------|
| inventory | total_videos, valid_videos, corrupt_videos, corrupt_paths, format_distribution, label_distribution, file_size_kb |
| spatial   | height, width, aspect_ratio, megapixels — each a stat dict                                                       |
| temporal  | frame_count, sampled_frames, fps, duration_sec, temporal_brightness_std, temporal_contrast_std                   |
| quality   | brightness_mean, brightness_std, contrast_mean,                                                                  |

|        |                                                                                                           |
|--------|-----------------------------------------------------------------------------------------------------------|
|        | sharpness_mean, sharpness_min, blur_fraction                                                              |
| motion | frame_diff_mean, motion_intensity_mean,<br>motion_blur_fraction, scene_change_count,<br>scene_change_rate |
| colour | rgb_mean_mean [R,G,B] in [0,1], rgb_std_mean [R,G,B]                                                      |
| labels | label_distribution dict, class_imbalance_ratio                                                            |

## VideoEDA.get\_record(index)

| Parameter | Type | Default | Description                                |
|-----------|------|---------|--------------------------------------------|
| index     | int  | 0       | Index into eda._records (order of loading) |

Returns: VideoRecord object.

## VideoEDA.get\_frames(index)

| Parameter | Type | Default | Description             |
|-----------|------|---------|-------------------------|
| index     | int  | 0       | Index into eda._records |

Returns: ndarray shape (T, H, W, 3), dtype uint8 — sampled frames.

## VideoEDA.temporal\_profile(index)

| Parameter | Type | Default | Description             |
|-----------|------|---------|-------------------------|
| index     | int  | 0       | Index into eda._records |

Returns: dict with keys brightness, contrast, sharpness (each ndarray of length T), and frame\_diff (ndarray of length T-1).

## VideoEDA.pairwise\_video\_distances(max\_videos)

| Parameter  | Type | Default | Description                                      |
|------------|------|---------|--------------------------------------------------|
| max_videos | int  | 50      | Maximum videos to include in the distance matrix |

Returns: (D, names) where D is a symmetric float ndarray of shape (N, N) and names is a list of N label/stem strings.

## VideoEDA.plot\_dataset()

| Parameter | Type     | Default | Description                                        |
|-----------|----------|---------|----------------------------------------------------|
| figsize   | tuple    | (24,24) | Figure dimensions in inches                        |
| save_path | str None | None    | Path to save the figure; None = show interactively |
| dpi       | int      | 150     | Resolution for saved figures                       |

## VideoEDA.plot()

| Parameter   | Type     | Default | Description            |
|-------------|----------|---------|------------------------|
| video_index | int      | 0       | Which video to display |
| figsize     | tuple    | (16,10) | Figure dimensions      |
| save_path   | str None | None    | Save path or None      |
| dpi         | int      | 150     | Resolution             |



## VideoEDA.plot\_motion\_profile()

| Parameter   | Type     | Default | Description         |
|-------------|----------|---------|---------------------|
| video_index | int      | 0       | Which video to plot |
| save_path   | str None | None    | Save path or None   |
| dpi         | int      | 150     | Resolution          |

## VideoEDA.plot\_pairwise\_video\_distances()

| Parameter | Type     | Default | Description       |
|-----------|----------|---------|-------------------|
| save_path | str None | None    | Save path or None |
| dpi       | int      | 150     | Resolution        |

## VideoEDA.plot\_videos\_grid()

| Parameter | Type     | Default | Description        |
|-----------|----------|---------|--------------------|
| n         | int      | 12      | Max videos to show |
| cols      | int      | 4       | Grid columns       |
| save_path | str None | None    | Save path or None  |
| dpi       | int      | 150     | Resolution         |

## VideoEDA.plot\_temporal\_statistics()

| Parameter   | Type     | Default | Description            |
|-------------|----------|---------|------------------------|
| video_index | int      | 0       | Which video to analyse |
| save_path   | str None | None    | Save path or None      |
| dpi         | int      | 150     | Resolution             |

## VideoEDA.plot\_quality\_summary()

| Parameter | Type     | Default | Description       |
|-----------|----------|---------|-------------------|
| save_path | str None | None    | Save path or None |
| dpi       | int      | 150     | Resolution        |

## VideoEDA.plot\_frame\_samples()

| Parameter   | Type     | Default | Description                 |
|-------------|----------|---------|-----------------------------|
| video_index | int      | 0       | Which video to sample from  |
| n           | int      | 12      | Number of frames to display |
| cols        | int      | 4       | Grid columns                |
| save_path   | str None | None    | Save path or None           |
| dpi         | int      | 150     | Resolution                  |

**VideoEDA.report()**

| Parameter   | Type | Default                  | Description                   |
|-------------|------|--------------------------|-------------------------------|
| output_path | str  | viseda_video_report.html | Path for the HTML output file |

## Chapter 10 — HTML Report, CLI, and Test Workflow

VideoEDA can be used from Python, from the package command-line interface, or through the project test script. These three entry points are related, but they are not identical. This chapter documents the current behaviour of `report()`, the `viseda video` command, and `testvideo.py` so that users can reproduce analyses without confusing package CLI arguments with test-suite arguments.

### 10.1 HTML Report Architecture

`VideoEDA.report(output_path)` calls `summary()` and writes a self-contained HTML file. The report uses responsive cards, horizontal distribution bars, and compact statistic rows. No external CSS or JavaScript files are required, so the generated report can be opened directly in a browser or shared as a single HTML file.

```
from viseda import VideoEDA

eda = VideoEDA(verbose=True)
eda.load("UCF-101/", label_from_parent=True)

# Write a standalone HTML report
report_path = eda.report("outputs_video/video_report.html")
print(report_path)
```

The HTML generator is intentionally summary-focused. It serialises the aggregate values returned by `summary()`; it does not embed Matplotlib dashboards or raw video frames.

### 10.2 HTML Report Sections

| HTML section              | Current contents                                                          |
|---------------------------|---------------------------------------------------------------------------|
| Header                    | Total videos, valid videos, and corrupt videos.                           |
| Inventory                 | Counts, label distribution, and format distribution.                      |
| Spatial                   | Height, width, and aspect-ratio summary statistics.                       |
| Temporal Frame Statistics | Frame count, FPS, duration, and temporal brightness standard deviation.   |
| Quality and Motion Blur   | Brightness, contrast, sharpness, blur fraction, and motion-blur fraction. |
| Motion                    | Frame difference, motion intensity, and scene-change rate.                |
| Footer                    | Generated-by attribution for VisEDA.                                      |

Each scalar card displays the available min, max, mean, median, standard deviation, 25th percentile, and 75th percentile values. Label and format counts are visualised as horizontal bars, scaled relative to the largest group.

### 10.3 What the HTML Report Does Not Include

The current report is deliberately smaller than `summary()` and `plot_dataset()`. The following items are available elsewhere in VideoEDA but are not currently rendered into the HTML file:

- The RGB aggregate values returned in `summary()['colour']`.
- The `class_imbalance_ratio` stored in `summary()['labels']`.
- The complete `corrupt_paths` list.
- Megapixels and `temporal_contrast_std` cards.
- Dataset scatter plots, pairwise-distance heatmaps, representative frames, or other Matplotlib figures.
- Per-video temporal profiles and raw sampled frames.

Note: Use `summary()` when you need the full nested statistics dictionary, and `plot_dataset()` when you need the comprehensive visual dashboard. The HTML report is a portable summary rather than a replacement for those outputs.

## 10.4 viseda video CLI Reference

The package CLI is defined by the `video` sub-command. It constructs VideoEDA, loads the supplied path, prints the JSON-formatted summary, and optionally writes a report or plot.

| Argument                             | Default  | Meaning                                                                                                       |
|--------------------------------------|----------|---------------------------------------------------------------------------------------------------------------|
| <code>path</code>                    | required | Single video file or dataset directory.                                                                       |
| <code>--max</code>                   | None     | Maximum number of videos to analyse.                                                                          |
| <code>--frame-sample-rate</code>     | 5        | Analyse every Nth frame.                                                                                      |
| <code>--max-frames-per-video</code>  | 300      | Maximum sampled frames retained per video.                                                                    |
| <code>--blur-threshold</code>        | 80.0     | Sharpness threshold used to compute <code>blur_fraction</code> .                                              |
| <code>--motion-blur-threshold</code> | 80.0     | Sharpness threshold used in the motion-blur rule.                                                             |
| <code>--label-from-parent</code>     | False    | Use each video's parent directory as its class label.                                                         |
| <code>--report</code>                | None     | Write the HTML report to this path.                                                                           |
| <code>--plot</code>                  | False    | Generate a VideoEDA dashboard.                                                                                |
| <code>--single-plot</code>           | False    | With <code>--plot</code> , use <code>plot()</code> for video index 0 instead of <code>plot_dataset()</code> . |
| <code>--save-plot</code>             | None     | Output path passed to the chosen plotting method.                                                             |

## 10.5 CLI Examples

```
# 1. Summarise a labelled directory
viseda video "C:/datasets/UCF-101" --label-from-parent

# 2. Summarise and save the HTML report
viseda video "C:/datasets/UCF-101" --label-from-parent --report outputs_video/video_report.html

# 3. Save the full dataset dashboard
viseda video "C:/datasets/UCF-101" --label-from-parent --plot --save-plot outputs_video/plot_dataset.png

# 4. Analyse a single file and save its single-video dashboard
viseda video "C:/videos/sample.mp4" --plot --single-plot --save-plot outputs_video/plot.png

# 5. Faster coarse sampling for a large dataset
viseda video "C:/datasets/large_video_set" --max 500 --frame-sample-rate 10 --max-frames-per-video 100
```

## 10.6 CLI vs Python API

The CLI exposes the most commonly used VideoEDA controls, but it does not expose every constructor parameter. In the current CLI, `scene_change_threshold` and `resize_width` retain their VideoEDA constructor defaults. Use the Python API when those values need to be changed.

```
from viseda import VideoEDA

eda = VideoEDA(
    frame_sample_rate=5,
    max_frames_per_video=300,
    blur_threshold=80.0,
    motion_blur_threshold=80.0,
```

```

    scene_change_threshold=20.0, # Python API only in current CLI
    resize_width=160,           # Python API only in current CLI
)
eda.load("dataset/", label_from_parent=True)

```

Note: Do not use `testvideo.py` options such as `--quick` or `--save-plots` with the `viseda video` command. Those switches belong to the test script, not the package CLI.

## 10.7 testvideo.py Workflow

`testvideo.py` is a validation script for the VideoEDA implementation. It is not the same interface as `viseda video`. The script first builds an eight-video in-memory synthetic fixture covering moving, flicker, blurred, and random scenes, then executes a sequence of API and plot checks. It subsequently tests generated video files, supported writer formats, and edge cases. Optional `--file` and `--dir` arguments add a final user-supplied-data check.

| Test stage                      | What is validated                                                                                         |
|---------------------------------|-----------------------------------------------------------------------------------------------------------|
| 1 · LOAD — Video arrays         | Eight synthetic NumPy videos loaded through <code>load_arrays()</code> .                                  |
| 2 · SUMMARY                     | Inventory, labels, frame count, sharpness, and motion-blur statistics.                                    |
| 3 · PER-RECORD FIELD VALIDATION | Sixteen required <code>VideoRecord</code> fields on valid records.                                        |
| 4 · TEMPORAL PROFILE            | Brightness-profile length matches <code>sampled_frames</code> .                                           |
| 5 · PAIRWISE VIDEO DISTANCES    | Square pairwise matrix and matching names.                                                                |
| 6 · PLOTS                       | Dataset, single-video, video-grid, motion-profile, and pairwise plots.                                    |
| 7 · HTML REPORT                 | <code>report()</code> creates the requested HTML file.                                                    |
| 8 · LOAD — Directory            | Synthetic MP4 directory and parent-folder labels when <code>VideoWriter</code> is available.              |
| 9 · FILE FORMATS                | Written MP4/AVI examples when the OpenCV writer is available.                                             |
| 10 · EDGE CASES                 | Pre-load <code>RuntimeError</code> , <code>max_videos</code> , single-frame handling, and invalid arrays. |
| 11/12 · USER DATA               | Optional user file or directory summary and plot checks.                                                  |

### testvideo.py arguments

| Argument                   | Meaning                                                                                                   |
|----------------------------|-----------------------------------------------------------------------------------------------------------|
| <code>--file PATH</code>   | Add a test using one user-supplied video file.                                                            |
| <code>--dir PATH</code>    | Add a test using a user-supplied video directory with parent-folder labels.                               |
| <code>--save-plots</code>  | Save test figures under <code>outputs_video</code> instead of showing them.                               |
| <code>--quick</code>       | Use coarser frame sampling in several test stages.                                                        |
| <code>--report PATH</code> | HTML filename used by the <code>test_report</code> stage; default <code>viseda_video_report.html</code> . |

```

# Run the full built-in validation suite
python testvideo.py --save-plots --quick

# Also check a user dataset directory
python testvideo.py --dir "C:/datasets/UCF-101" --save-plots --quick

# Also check one user video file
python testvideo.py --file "C:/videos/sample.mp4" --save-plots

```

## 10.8 Test-Script Outputs and Report Behaviour

When `--save-plots` is used, the current test suite writes the five plotting methods exercised by `test_plots()` to `outputs_video`: `plot_dataset.png`, `plot.png`, `plot_videos_grid.png`, `plot_motion_profile.png`, and `plot_pairwise_video_distances.png`. VideoEDA itself also exposes `plot_temporal_statistics()`, `plot_quality_summary()`, and `plot_frame_samples()`, although the current `test_plots()` function does not call those three methods.

A particularly important detail is the current `--report` workflow. `test_report()` is executed on the initial eight-video synthetic array fixture before the optional user-file or user-directory branches run. The current user branches call `summary()` and optionally save a plot, but they do not regenerate the report. Therefore, when this exact `testvideo.py` is run with `--dir` or `--file`, the HTML file named by `--report` still describes the synthetic eight-video fixture. For a report of the real dataset, call `VideoEDA.report()` directly or use the `visedata video --report` CLI path.

```
# Recommended way to report the actual user dataset
from viseda import VideoEDA

eda = VideoEDA(verbose=True, frame_sample_rate=5)
eda.load("C:/datasets/UCF-101", label_from_parent=True)
eda.report("outputs_video/ucf101_report.html")
```

Note: This documentation describes the attached current `testvideo.py` exactly. If the test script is later changed to regenerate the report inside `test_user_file()` or `test_user_dir()`, update this section accordingly.

## 10.9 Troubleshooting

| Symptom                                                       | Likely cause                                                                     | Action                                                                                                                                                                   |
|---------------------------------------------------------------|----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ImportError: VideoEDA requires OpenCV                         | opencv-python is not installed for file-based video loading.                     | Run <code>pip install opencv-python</code> . Array-based <code>load_arrays()</code> does not require file decoding.                                                      |
| No valid videos to plot                                       | All resolved files failed decoding or the path contains no supported extensions. | Check the input path, codecs, and <code>SUPPORTED_EXTS</code> .                                                                                                          |
| Label distribution is empty                                   | No labels were provided.                                                         | Use <code>label_from_parent=True</code> / <code>--label-from-parent</code> or pass a labels mapping.                                                                     |
| Pairwise distance raises <code>ValueError</code>              | Fewer than two valid videos are loaded.                                          | Load at least two valid videos before requesting the matrix.                                                                                                             |
| VideoWriter unavailable in tests                              | The local OpenCV build cannot create the requested MP4/AVI writer.               | The directory/format tests are designed to skip when the writer cannot be opened.                                                                                        |
| HTML report shows 8 synthetic videos after <code>--dir</code> | The current test suite writes the report before the user-directory branch.       | Generate the real report with <code>eda.report()</code> or <code>visedata video --report</code> .                                                                        |
| Dashboard analysis is slow                                    | Too many frames or large resized frames are being processed.                     | Increase <code>frame_sample_rate</code> , reduce <code>max_frames_per_video</code> , set <code>max_videos</code> , or use a smaller <code>resize_width</code> in Python. |
| Saved PDF/DOCX documentation is locked                        | The output is open in another application.                                       | Close the existing file; the documentation generator can fall back to timestamped output names.                                                                          |

# Chapter 11 — Roadmap

## Version 1.0.0 — Initial Release

- VideoEDA class with full per-video statistical analysis.
- `load()` for file-based loading (mp4, avi, mov, mkv, webm, mpeg, m4v).
- `load_arrays()` for NumPy array inputs without requiring OpenCV.
- All quality metrics: brightness, contrast, sharpness, blur\_fraction.
- All motion metrics: frame\_diff, motion\_intensity, motion\_blur\_fraction, scene\_change\_count/rate.
- Temporal consistency metrics: temporal\_brightness\_std, temporal\_contrast\_std.
- Pairwise video distance computation and heatmap visualisation.
- Eight plot methods: `plot_dataset`, `plot`, `plot_motion_profile`, `plot_pairwise_video_distances`, `plot_videos_grid`, `plot_temporal_statistics`, `plot_quality_summary`, `plot_frame_samples`.
- HTML report generation via `report()`.
- CLI integration via `viseda video` sub-command.

## Roadmap

| Version | Planned Feature                                                            |
|---------|----------------------------------------------------------------------------|
| v1.2.0  | Optical flow-based motion estimation (more accurate than frame difference) |
| v1.2.0  | Audio analysis module (silence detection, audio quality metrics)           |
| v1.2.0  | Perceptual hash-based near-duplicate detection across videos               |
| v1.3.0  | Interactive Plotly dashboards as alternative to Matplotlib                 |
| v1.3.0  | Per-frame object detection counts (using lightweight detector)             |
| v1.3.0  | Temporal FFT analysis (flicker frequency detection)                        |
| v1.4.0  | GPU-accelerated frame processing via CuPy or PyTorch                       |
| v1.4.0  | Video embedding and UMAP/t-SNE visualisation of dataset clusters           |
| v1.5.0  | Depth video / RGB-D EDA module                                             |

## Appendix A — VideoRecord Fields

Complete field reference for the VideoRecord object:

| Field                   | Type       | Description                                            |
|-------------------------|------------|--------------------------------------------------------|
| path                    | str        | File path or " for array inputs                        |
| label                   | str None   | Class label if provided                                |
| file_ext                | str        | File extension or 'array'                              |
| file_size_kb            | float None | File size in KB                                        |
| is_corrupt              | bool       | True if the video could not be processed               |
| error                   | str None   | Error message string if is_corrupt=True                |
| n_frames                | int        | Total frames in the video container                    |
| sampled_frames          | int        | Number of frames actually sampled                      |
| fps                     | float      | Frames per second from container metadata              |
| duration_sec            | float      | n_frames / fps                                         |
| height                  | int        | Frame height after optional resize                     |
| width                   | int        | Frame width after optional resize                      |
| channels                | int        | Colour channels (always 3 after normalisation)         |
| aspect_ratio            | float      | width / height                                         |
| megapixels              | float      | height * width / 1,000,000                             |
| brightness_mean         | float      | Mean of per-frame mean luminance (0-255)               |
| brightness_std          | float      | Std of per-frame brightness across frames              |
| contrast_mean           | float      | Mean of per-frame luminance std                        |
| contrast_std            | float      | Std of per-frame contrast                              |
| sharpness_mean          | float      | Mean Laplacian variance across frames                  |
| sharpness_std           | float      | Std of per-frame sharpness                             |
| sharpness_min           | float      | Minimum per-frame sharpness (worst frame)              |
| blur_fraction           | float      | Fraction of frames with sharpness < blur_threshold     |
| frame_diff_mean         | float      | Mean absolute frame-to-frame pixel difference          |
| frame_diff_std          | float      | Std of frame differences                               |
| motion_intensity_mean   | float      | Same as frame_diff_mean                                |
| motion_intensity_std    | float      | Same as frame_diff_std                                 |
| motion_blur_fraction    | float      | Fraction of transitions with high diff + low sharpness |
| scene_change_count      | int        | Number of frame diffs above scene_change_threshold     |
| scene_change_rate       | float      | scene_change_count / (sampled_frames - 1)              |
| temporal_brightness_std | float      | Std of per-frame mean brightness over the video        |
| temporal_contrast_std   | float      | Std of per-frame contrast over the video               |
| rgb_mean                | ndarray(3) | Mean R,G,B pixel values in [0,1]                       |
| rgb_std                 | ndarray(3) | Std R,G,B pixel values                                 |
| sample_frames           | ndarray    | Up to 6 preview frames, shape (N,H,W,3) uint8          |



## Appendix B — Glossary

---

### Laplacian

A second-order spatial derivative filter applied to grayscale image frames. Its variance (Laplacian variance) measures image sharpness — sharp images have many strong edges and high Laplacian variance; blurry images have few edges and low variance.

### Frame difference

The mean absolute pixel difference between two consecutive sampled frames. The primary measure of motion intensity in VideoEDA.

### Motion blur

Blur caused by camera or subject movement during frame exposure. In VideoEDA, a frame is classified as motion-blurred when frame difference is in the top 25% AND sharpness is below `motion_blur_threshold`.

### Scene change

An abrupt transition between visually dissimilar frames, typically caused by a video cut, fade, or wipe. Detected when `frame_diff > scene_change_threshold`.

### Temporal brightness std

Standard deviation of per-frame mean brightness across the sampled frames of a video. Measures lighting stability. High values indicate flicker, AGC activity, or scene transitions.

### ITU-R BT.601

The standard formula for converting RGB to luminance:  $L = 0.299R + 0.587G + 0.114B$ , reflecting human visual sensitivity to each colour channel.

### `frame_sample_rate`

The frame sampling interval. A value of `N` means every `N`th frame is sampled. Controls the trade-off between analysis speed and temporal resolution.

### Pairwise distance

The Euclidean distance between two videos in a normalised 12-dimensional feature space. Used to measure dataset diversity and detect near-duplicate videos.

### Blur fraction

The proportion of a video's sampled frames that are classified as blurry (Laplacian variance below `blur_threshold`). Ranges from 0.0 (no blurry frames) to 1.0 (all blurry).

### Data leakage

When information from the validation or test set influences training, typically through near-duplicate videos that appear in both splits. The pairwise distance heatmap helps detect this.

## Appendix C — Acknowledgements

---

VideoEDA builds on the excellent work of the following open-source projects:

- OpenCV (cv2): Video file reading, frame decoding, colour conversion, Laplacian computation, and frame resizing. The backbone of all file-based video I/O in VideoEDA.
- NumPy: All numerical array operations, statistical computations, and the frame difference calculations.
- Matplotlib: All visualisations — dashboards, time series plots, heatmaps, histograms, and scatter plots.
- The UCF-101 Action Recognition dataset (Soomro et al., 2012) was used as the primary real-world test dataset for validating VideoEDA. Available at: <https://www.crcv.ucf.edu/data/UCF101.php>